Davante

5

PROGRAMACIÓN

Recursividad

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Conceptos básicos de la recursividad 5

2.1. Caso base y recursividad 5

2.2. Consideraciones al utilizar la recursividad 5

2.3. Ventajas y desventajas de la recursividad 6

/ 3. Desarrollo de soluciones recursivas 7

/ 4. Ejemplos de funciones recursivas 8

4.1. Factorial 8

4.2. Serie de Fibonacci 9

/ 5. Caso práctico 1: "Torres de Hanoi" 10

/ 6. Coste de una solución recursiva 11

6.1. Esfuerzo de desarrollo 11

6.2. Complejidad temporal 11

6.3. Complejidad espacial 11

/ 7. Tipos de recursividad 12

7.1. Recursividad directa 12

7.2. Recursividad indirecta 12

7.3. Recursividad en aumento 13

7.4. Recursividad múltiple 13

ÍNDICE

/ 8. Caso práctico 2: "Coste desorbitado" 13

/ 9. Robustez de la recursividad elemental: Suma 14

9.1. Uso de drivers para el método recursivo 15

/ 10. Resumen y resolución del caso práctico de la unidad 17

/ 11. Bibliografía 18

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de recursividad.

Saber utilizar algoritmos recursivos.

Comprender el funcionamiento de la recursividad compleja.

Saber aplicar diferentes tipos de recursividad.

/ 1. Introducción y contextualización práctica

Durante el desarrollo de algoritmos, es importante tener en cuenta diferentes estrategias para poder resolver un problema determinado. Normalmente, cuando intentamos solucionar un problema a través de un algoritmo, no existe una única solución posible. De esta forma, existen soluciones que son mejores, en un contexto determinado, que otras. Es responsabilidad del programador elegir correctamente la que más se adapte al contexto en el que se encuentra.

Como norma general, los programadores tienden a implementar soluciones iterativas que pueden ser realmente complejas de aplicar. En muchas ocasiones, aplicar una solución recursiva puede aligerar el proceso de desarrollo y mejorar el rendimiento del algoritmo. Hay que prestar especial atención a la hora de crear soluciones recursivas, porque podemos caer en un error fácilmente.

Depurar las soluciones recursivas puede ser, en ocasiones, complejo, pues va a depender del tipo de recursividad que estemos aplicando.

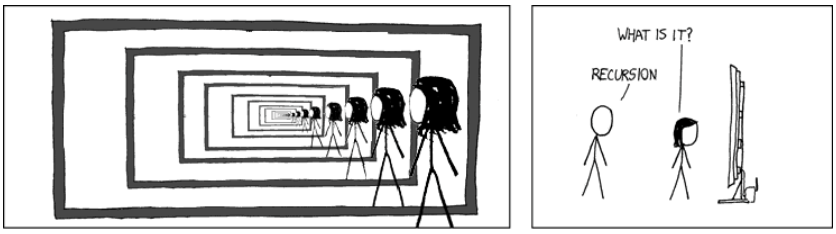

Fig.1. Recursividad.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Fallo crítico durante la ejecución"

https://on.soundcloud.com/ySCy8UVUbdExTcUEA

/ 2. Conceptos básicos de la recursividad

A la hora de crear un programa informático, compuesto por múltiples algoritmos, es posible utilizar diferentes estrategias para la implementación de estos algoritmos.

Se pueden emplear soluciones iterativas que se acercan más al pensamiento humano que otro tipo de soluciones. Una alternativa a la iteración es la recursividad. Podríamos definir formalmente la recursividad como la **utilización de un concepto o proceso para la resolución de problemas sobre sí mismo mediante una versión más sencilla**. La recursividad es un término heredado de la matemática que se utiliza también en el mundo de la computación.

Desde el punto de vista de la informática, podemos decir que **la recursividad está relacionada con la utilización de funciones y procedimientos**. Aunque conceptualmente es fácilmente detectable, en la práctica, cuando el tamaño de los algoritmos aumenta significativamente, puede ser complicado detectar que estamos utilizando soluciones recursivas. En el desarrollo de algoritmos, para implementar soluciones recursivas, únicamente es necesario utilizar funciones y procedimientos que realicen llamadas sobre sí mismos. Técnicamente, la utilización de la recursividad no es compleja, sin embargo, puede ser complicado encontrar las condiciones de parada de la recursividad.

2.1. Caso base y recursividad

Para poder implementar soluciones recursivas, es necesario detectar diferentes situaciones en los algoritmos que permitan implementar dicha recursividad. De esta forma, podemos diferenciar los siguientes elementos:

- **Caso base**: Se trata de la situación de finalización de la recursividad. En este caso, podemos identificar un valor por defecto de una función.
- **Proceso de recursión**: Durante el proceso de recursividad se realizan múltiples llamadas de una función o un procedimiento sobre sí mismo, hasta que llegamos al caso base.
- **Resolución final**: Cuando el proceso de recursividad llega al caso base, se comienzan a resolver todos los valores intermedios hasta proporcionar el resultado final.

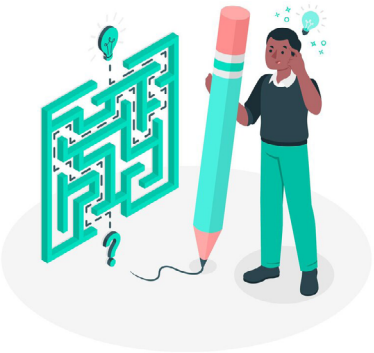

Fig.2. Búsqueda del caso base de una solución recursiva.

2.2. Consideraciones al utilizar la recursividad

Al realizar una llamada recursiva, se ponen en marcha diferentes procedimientos que es importante tener en cuenta para evitar problemas en tiempo de ejecución.

Cuando realizamos una llamada a una función, sea recursiva o no, se crean estructuras intermedias para poder atender a la llamada correspondiente. Una de las estructuras que destaca por encima del resto es la **pila de llamadas o *stack***. Dependiendo de la memoria que tenga la máquina en la que estamos ejecutando nuestro algoritmo, dispondremos de un mayor número de llamadas posible.

De esta forma, a mayor tamaño de memoria, mayor número de llamadas es soportado por la máquina. Cuando realizamos muchas llamadas a funciones y procedimientos, se puede producir un error conocido como **desbordamiento de pila o *stack overflow***. Este desbordamiento tiene que ver con los recursos de la máquina, en concreto, con la memoria disponible y con la secuencia de llamadas que se ha realizado.

Cuando implementamos soluciones recursivas, es posible caer fácilmente en soluciones que nos llevan a desbordamientos de la pila. En este caso, habría que analizar el algoritmo para encontrar posibles errores en su implementación.

Un error muy típico es la **selección incorrecta de la condición de parada del algoritmo**. Por este motivo, hay que prestar especial atención a la definición del caso base, pues nos va a determinar cuándo podemos parar la recursividad. Si la condición de parada no está bien definida, se pueden producir diferentes situaciones.

¡! Sabías que...

La recursividad es una de las herramientas más potentes que nos podemos encontrar a la hora de generar soluciones *software*. Existen lenguajes que únicamente permiten la utilización de recursividad y carecen de estructuras iterativas.

Por ejemplo, podemos encontrarnos con:

- Un desbordamiento de pila.
- La utilización de memoria innecesaria.
- El bloqueo del algoritmo.
- Que dejemos la máquina en una situación inestable.

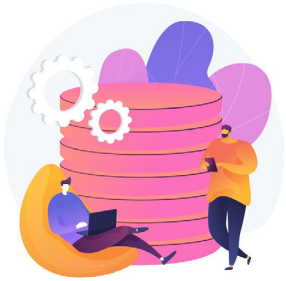

Fig.3. Protecciones para evitar desbordamientos de pila.

Podríamos decir que el desbordamiento de pila equivaldría a una solución iterativa donde se produce un bucle infinito.

2.3. Ventajas y desventajas de la recursividad

Una de las primeras preguntas que nos surge cuando vamos a utilizar la recursividad es si realmente estamos introduciendo una ventaja algorítmica en la implementación de nuestra solución. **Como norma general**, podemos decir que **la implementación de soluciones recursivas es una opción más elegante de resolución y, en muchas ocasiones, más eficiente**. Podríamos utilizar soluciones recursivas fácilmente cuando empleamos el mismo problema con diferentes valores de entrada. Sin embargo, **existen situaciones en las que el uso de soluciones recursivas puede ser una forma más lenta y menos eficiente de llevar a cabo una determinada implementación**.

Si tenemos en cuenta diferentes estructuras de datos, la utilización de recursividad suele ser una **buena opción** cuando tenemos que recorrer **estructuras de tipo arbóreo**. En este tipo de estructuras, normalmente realizamos las mismas operaciones en cada nodo del árbol hasta llegar a los elementos hojas. Gracias a la recursividad, cuando nos enfrentamos a estructuras de este tipo, es posible realizar diferentes llamadas fácilmente.

Cuando implementamos soluciones recursivas, no es necesario especificar exactamente toda la secuencia de pasos que hay que realizar para solventar un problema.

Es importante que consideremos que una llamada recursiva realiza siempre los mismos pasos cambiando únicamente los valores de entrada en cada llamada recursiva hasta llegar al caso base. En muchas ocasiones, se opta por soluciones recursivas porque la implementación mediante soluciones iterativas sería muy compleja de realizar, siendo realmente trivial con soluciones recursivas.

Fig.4. Ventajas y desventajas de la recursividad.

Existen algunas **desventajas** cuando implementamos soluciones recursivas. La más destacable de todas es que muchas veces la implementación recursiva a la que llegamos es ineficiente, pues genera **gran número de llamadas** para llegar a la solución del problema. Además, desde el punto de vista de la pila, se van creando diferentes instancias, con valores duplicados, de modo que se tienen variables duplicadas en cada llamada. Por todo ello, en ocasiones, se realiza un **abuso de uso de memoria** que puede afectar al rendimiento de la máquina sobre la que se está ejecutando nuestro algoritmo.

Recuerda...

Es fundamental poder establecer el caso base para poder devolver un valor en el proceso de recursión. Si se produce un error a la hora de determinar el caso base, los valores que vamos a generar van a ser erróneos.

Audio 1. "Error indetectable"
https://on.soundcloud.com/KRi7A7MF8dKK3muy9

/ 3. Desarrollo de soluciones recursivas

Para desarrollar soluciones recursivas, es necesario poder detectar los elementos básicos de la recursividad: **caso base y proceso de recursión**. En ocasiones, es muy sencillo detectar la recursividad, porque podemos basarnos en la matemática subyacente. Esto quiere decir que muchas veces las soluciones que estamos implementando están relacionadas directamente con el mundo de la matemática. Si nos centramos en el caso de la matemática, podemos llegar a un proceso de recursividad fácilmente.

Es posible realizar un análisis de las funciones recursivas utilizando el **diseño descendente**, en el que dividimos un problema mayor en subproblemas de tamaño más pequeño. Si tenemos en cuenta este proceso de diseño, la utilización de la recursividad está directamente relacionada, porque realizamos llamadas cada vez con valores más pequeños hasta llegar al valor que resuelve el problema, el caso base.

Es importante tener en cuenta que toda solución recursiva, para que esté bien formulada, debe satisfacer las condiciones vistas anteriormente de **definición del caso base y del caso inductivo**. Aunque es posible utilizar funciones recursivas en el contexto matemático, la implementación de dichas soluciones no tiene que estar directamente relacionada con la matemática, pues se puede aplicar a problemas de diferente contexto.

Para poder diseñar soluciones recursivas de un problema, la mejor estrategia que podemos seguir es la **simplicidad**. Esto quiere decir que debemos traducir el problema a un algoritmo manteniendo siempre las reglas del lenguaje que estamos utilizando en cada momento. Por ejemplo, no es lo mismo realizar procesos recursivos en un lenguaje tipo C que en un lenguaje tipo Java. **Cada lenguaje de programación dispone de unas normas y reglas que hay que seguir para poder realizar llamadas recursivas**.

Normalmente, esto significa que podemos tener un procedimiento, una función cuya estructura general será la determinada por las reglas de la recursividad: caso general y caso base.

Fig.5. Desarrollo de soluciones recursivas.

/ 4. Ejemplos de funciones recursivas

Tradicionalmente, podemos diferenciar algunas de las funciones típicas utilizadas en la comprensión y explicación de la recursividad.

4.1. Factorial

El **cálculo factorial de un número determinado es un ejemplo típico de recursividad**. Desde el punto de vista matemático, podemos definir el factorial de un número de la siguiente forma:

```
F(n) =
        1 si  n = 0
        n*f(n-1) si n <> 0
```

Fácilmente, se puede observar que la propia definición de la función matemática factorial implica una recursividad. Podemos diferenciar, en esta definición, dos partes: caso base y caso inductivo. En el caso base, se establece que la función devolverá 1 si N = 0. En el caso inductivo, es necesario calcular el resto de los valores factoriales para poder resolver la llamada función.

Para implementar una solución recursiva, la forma más sencilla es realizando una traducción directa, teniendo en cuenta las normas del lenguaje de programación.

Desde nuestro punto de vista, vamos a considerar un pseudolenguaje de programación. De esta forma, la implementación recursiva para el número factorial sería:

```
funcion factorial(n)
  si n=1 entonces
    devolver 1
  sino
    devolver n * factorial(n-1)
fin funcion
```

Como se puede observar en la implementación y en la imagen, existe una traducción directa de cada uno de los dos casos. Se puede diferenciar el caso base del caso general mediante una estructura y expresión condicional. De esta forma, si se cumple el caso base, se devolverá el valor 1.

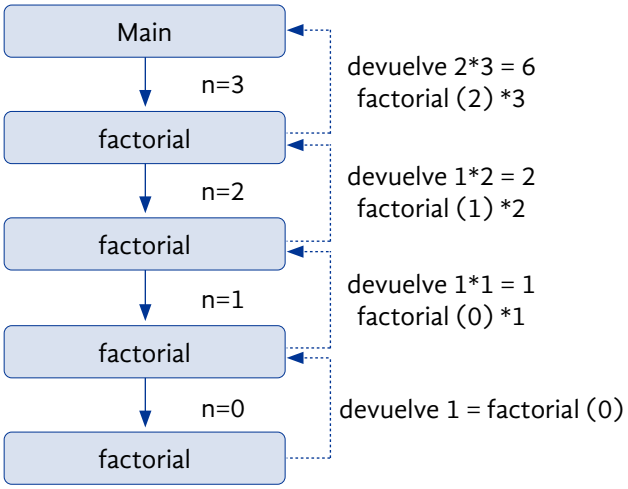

Fig.6. Funcionamiento interno de las llamadas recursivas.

4.2. Serie de Fibonacci

La función de Fibonacci es una de las más típicas utilizadas en la explicación de la recursividad. Desde el punto de vista matemático, representa una espiral que, se ha observado, se cumple en diferentes ámbitos de la naturaleza.

Desde el punto de vista matemático, podemos expresar la función de Fibonacci de la siguiente forma:

```
F(n) =
        0 si n = 0
        1 si n = 1
        F(n-1) + f(n-2) si n >= 2
```

Como se puede observar en la expresión matemática, disponemos de dos casos bases en los que se diferencian dos valores para N=0 y N=1. Uno de los problemas típicos que nos encontramos a la hora de resolver la función de Fibonacci mediante la recursividad es el desbordamiento de la pila. Esto se debe a que la función implica una doble recursividad en el caso general. Como se puede observar, se realiza una llamada recursiva para N-1 y N-2.

A continuación, se muestra un ejemplo de implementación de la función de Fibonacci mediante un algoritmo recursivo:

```
funcion fibonacci(n){
        si n == 0 {
                devolver 0
        } sino {
                si n == 1 {
                        devolver 1
                }sino {
                        devolver fibonacci (n-1)+fibonacci(n-2)
                }
}
```

Tal como se puede ver en la implementación del código recursivo, existe una traducción directa de la función matemática a la implementación del algoritmo de Fibonacci de forma recursiva. Además, se puede observar que, utilizando diferentes estructuras de selección, podemos diferenciar entre cada uno de los casos bases y el caso general.

Vídeo 1. "Desbordamiento de pila en ejecución"

https://vimeo.com/950715402/3896bdc39a

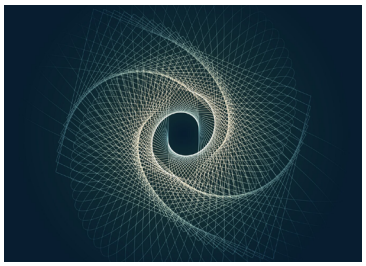

Fig.7. Representación gráfica de la función de Fibonacci.

/ 5. Caso práctico 1: "Torres de Hanoi"

Planteamiento: El problema de las torres de Hanoi consiste en mover una serie de discos de un poste a otro utilizando un tercer poste auxiliar.

Los discos que se encuentran en el primer poste poseen diferente tamaño y están apilados correctamente, es decir, de mayor a menor. Para poder solventar este problema, hay que seguir una serie de reglas en cada uno de los movimientos. Por ejemplo, solo es posible realizar el movimiento de un disco a la vez. En ninguno de los movimientos, un disco de menor tamaño puede estar debajo de un disco de mayor tamaño.

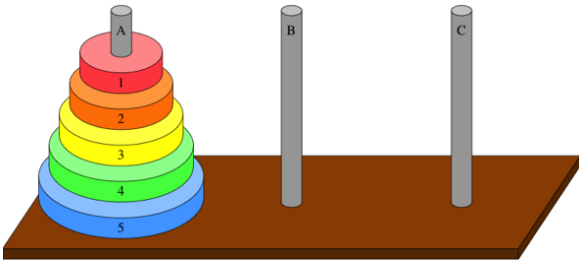

Fig.8. Torres de Hanoi.

Nudo: Si bien es cierto que es posible solucionar este problema utilizando una solución iterativa, el mismo es un ejemplo típico de problema que se suele resolver aplicando recursividad.

Desenlace: A continuación se muestra, en la imagen, el código realizado en su resolución:

```java
public class TorresDeHanoi {

    // Método recursivo para resolver las Torres de Hanoi
    public static void resolverHanoi(int numDiscos, char origen, char auxiliar, char destino) {
        if (numDiscos == 1) {
            // Caso base: solo queda un disco por mover
            System.out.println("Mover disco 1 de " + origen + " a " + destino);
            return;
        }

        // Mover los n-1 discos del origen al auxiliar usando el destino como apoyo
        resolverHanoi(numDiscos - 1, origen, destino, auxiliar);

        // Mover el disco n del origen al destino
        System.out.println("Mover disco " + numDiscos + " de " + origen + " a " + destino);

        // Mover los n-1 discos del auxiliar al destino usando el origen como apoyo
        resolverHanoi(numDiscos - 1, auxiliar, origen, destino);
    }

    public static void main(String[] args) {
        int numDiscos = 3; // Puedes cambiar este número para probar con más discos
        resolverHanoi(numDiscos, 'A', 'B', 'C');
    }
}
```

Fig.9. Resolución de las torres de Hanoi.

Caso base: Si numDiscos es 1, simplemente se mueve ese disco del poste de origen al poste de destino.

Caso recursivo: Si numDiscos es mayor que 1, el problema se descompone en tres pasos:

- Mover numDiscos-1 discos del origen al poste auxiliar.
- Mover el disco más grande (el disco numDiscos) del origen al destino.
- Mover los numDiscos-1 discos del poste auxiliar al destino.

/ 6. Coste de una solución recursiva

A la hora de resolver un algoritmo, es necesario tener en cuenta el coste que nos puede llevar la implementación de la solución elegida.

En ocasiones nos pedirán que la ejecución de un algoritmo esté acotada en tiempo mediante un requisito no funcional.

Recuerda...

Si tenemos diferentes estrategias para solucionar un problema, es decir, podemos elegir entre soluciones iterativas y recursivas, habría que evaluar cuál de ellas tiene mejor rendimiento en un problema determinado.

En otras ocasiones, nuestra empresa nos pedirá que el tiempo que nos lleve implementar esa solución no sea excesivo, pues estaríamos incurriendo en costes extras en el desarrollo del componente en cuestión. Cuando nos vamos a centrar en medir el coste de un algoritmo, es importante tener en cuenta las **métricas** que podamos usar. Definimos una métrica como una herramienta que nos permite medir determinados valores.

6.1. Esfuerzo de desarrollo

El esfuerzo de desarrollo es un coste que debemos tener en cuenta a la hora de implementar una solución. Si bien es cierto que la solución recursiva es muy elegante, muchas veces llegar a ella puede requerir suficiente tiempo como para desechar este camino. La justificación puede venir por el **tiempo que tenemos que invertir en detectar e implementar** dicha solución. La implementación de **una solución recursiva** puede ser compleja, llevando un tiempo significativo en su implementación.

6.2. Complejidad temporal

Al hacer referencia a la complejidad temporal, tenemos que considerar el tiempo que vamos a invertir en algoritmos para llegar a una solución determinada. Normalmente, existe una rama de la algoritmia que se encarga de estudiar la complejidad de los algoritmos. Sin embargo, desde el punto de vista de los algoritmos recursivos, es importante tener en cuenta el t**iempo que va a invertir el algoritmo en llegar a la solución**. Tenemos que desechar soluciones recursivas si tenemos unos límites temporales muy exigentes y no somos capaces de cubrirlos con nuestra solución recursiva.

6.3. Complejidad espacial

Hace referencia a los diferentes elementos que son necesarios para implementar la solución recursiva. Normalmente, está relacionada con los **recursos *hardware* de la máquina**. En concreto, podemos destacar el volumen de memoria necesario para hacer ejecutar una solución recursiva.

Fig.10. Optimización de costos en la implementación.

/ 7. Tipos de recursividad

Hasta ahora, hemos visto un único tipo de recursividad que nos permite implementar diferentes soluciones en nuestros algoritmos. Sin embargo, podemos diferenciar diferentes clases de recursividad que se adapten a cada una de las soluciones que queremos implementar.

7.1. Recursividad directa

Se trata de un tipo de recursividad en el que **la función o procedimiento únicamente realiza una llamada recursiva a sí misma**. Por ejemplo, en el cálculo del factorial de un número, realizamos una única llamada recursiva. El pseudocódigo sería el siguiente:

```
tipo nombre_método(int a, int b)
        inicio
        si (b = a)
                devuelve a
        sino
                devuelve nombre_método(a, b*5)
        fin
```

7.2. Recursividad indirecta

Es un tipo de recursividad que puede ser complicada de detectar, porque **se realizan múltiples llamadas a diferentes métodos y** en un punto determinado de las llamadas **se regresa de nuevo a la llamada original estableciendo un proceso recursivo**.

Vamos a partir de un método m1, que invoca al método m2, y este a su vez llama al método m3, el cual finaliza llamando al método m1. Este tipo de recursividad puede tener **varios niveles de profundidad**.

En el ejemplo expuesto, hemos terminado con una profundidad de 3, pero podría darse el caso de que m3 llamase a un nuevo método m4 y este a su vez a un método m5 que llamaría al método m1, obteniendo una profundidad de 5.

En la siguiente imagen, se muestra un ejemplo de recursividad indirecta donde se ilustra, con una línea continua, la llamada al método, y con una línea discontinua, el retorno.

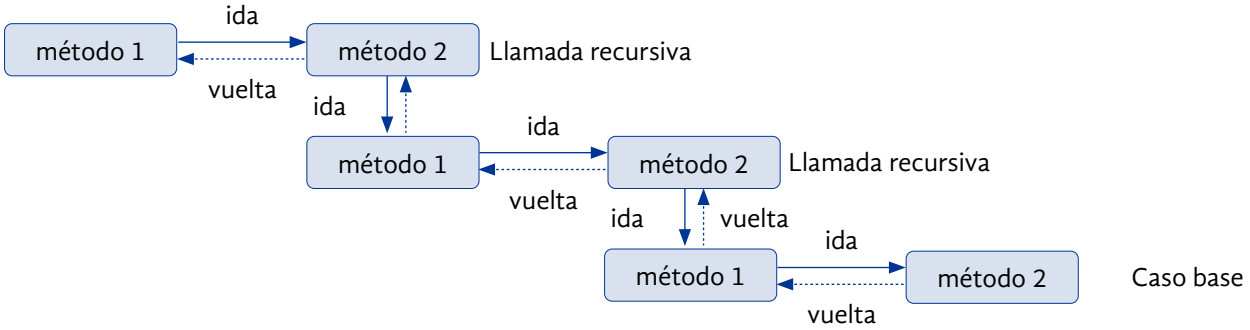

Fig.11. Esquema de llamadas recursivas en recursividad indirecta.

7.3. Recursividad en aumento

Se trata de un tipo de recursividad en la que se crean **operaciones que tienen que realizarse después de que se efectúe la última llamada recursiva**. Una de las características de este tipo de recursividad es que se realizan **llamadas diferidas** en el tiempo.

El pseudocódigo de la recursividad en aumento se muestra a continuación:

```
tipo nombre_método(int a)
        inicio
        si (a = 3)
                devuelve a
        sino
                devuelve a + nombre_método(a - 2)
        fin
```

7.4. Recursividad múltiple

Cuando utilizamos recursividad múltiple, estamos realizando **más de una llamada recursiva en el cuerpo de la función**. Por ejemplo, cuando empleamos una implementación recursiva para resolver el problema de Fibonacci, estamos usando recursividad múltiple, porque, en el caso general, se realiza más de una llamada recursiva.

Es importante tener en cuenta que, cuando realizamos recursividad múltiple, podemos tener un desbordamiento de la pila antes de lo esperado. Esto es porque el aumento suele estar relacionado con la recursividad múltiple, pues no tenemos en cuenta que cada llamada recursiva significa más de una llamada en la realidad. Caso práctico 2: "Coste desorbitado"

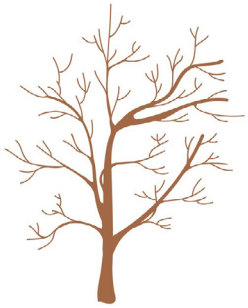

Fig.12. Ramas de la recursividad múltiple.

En el caso del cálculo de Fibonacci, podemos observar que se realizan dos llamadas recursivas en cada caso general. Puede haber situaciones en las que se efectúen más de dos llamadas recursivas.

Hay que prestar especial atención a este tipo de situaciones, porque nos puede llevar a un desbordamiento de la pila muy tempranamente.

```
tipo nombre_método(int a, int b)
        inicio
        si (b = 0)
                devuelve 1
        sino
                devuelve nombre_método(a-1, b) + nombre_
método(a-1, b-1)
        fin
```

/ 8. Caso práctico 2: "Coste desorbitado"

Planteamiento: Cuando recibimos los requisitos del equipo de análisis, observamos que tenemos muchas situaciones en las que podemos utilizar soluciones recursivas. Tras una implementación de un prototipo, observamos que el tiempo de ejecución del programa es muy alto. Existen situaciones en llamadas a funciones donde se retrasa mucho la devolución de un resultado. En este contexto, nuestro responsable nos pregunta por qué hemos implementado de esta forma alguna de las llamadas. Inmediatamente, observa que tenemos muchas llamadas recursivas y nos pide realizar determinados cambios para utilizar soluciones iterativas que mejoren el rendimiento.

Nudo: ¿Crees que tiene sentido lo que nos proponen?

Desenlace: En ocasiones, cuando utilizamos una solución recursiva, se puede producir una degradación significativa del rendimiento. Aunque se trata de una solución muy elegante a la hora de la implementación, al tener que ser traducida a un *hardware* para su ejecución, puede tener un rendimiento realmente bajo. Además, muchas de las llamadas recursivas que podemos implementar pueden estar limitadas por los parámetros de entrada. Existen situaciones en las que, cuando aumentamos los valores de entrada, el algoritmo recursivo se puede volver inestable y quedar bloqueado. Como se nos plantea en el caso, sería necesario poder explorar diferentes soluciones iterativas para aquellos puntos en los que el rendimiento ha decaído significativamente.

Fig.13. Determinadas implementaciones pueden aumentar los costes de desarrollo.

Por ello, lo que nos comenta nuestro responsable tiene todo el sentido, pues premia más mejorar el rendimiento que la elegancia de una solución algorítmica. Desde el punto de vista del usuario, la implementación que se ha realizado es transparente, es decir, el usuario ignora la elegancia de la solución, lo único que busca es que el programa funcione correctamente.

En el caso del cálculo de Fibonacci, podemos observar que se realizan dos llamadas recursivas en cada caso general. Puede haber situaciones en las que se efectúen más de dos llamadas recursivas.

Hay que prestar especial atención a este tipo de situaciones, porque nos puede llevar a un desbordamiento de la pila muy tempranamente.

/ 9. Robustez de la recursividad elemental: Suma

A continuación se va a realizar el algoritmo de la suma de los primeros n números mediante recursividad basándonos en el principio de inducción, partiendo de función suma de los n enteros como s(n). De esta manera, obtenemos:

- Caso base: s(1) = 1
- Caso inductivo s(n) = s(n-1) + n

Por tanto, el algoritmo para obtener la suma de manera recursiva sería:

```
public static int s(int n) {
        int valor;
        if (n == 1){
                valor = 1;
        }
        else {
                valor = s(n-1) + n;
        }
}
```

Fig.14. Algoritmo 1. Suma recursiva.

Al llamar a este algoritmo con un valor n = 1, se llama al caso base, que nos devuelve un 1 y, en caso contrario, donde n es distinto de 1, se llama al caso inductivo volviendo a llamar al método con n-1. En este ejemplo se ve claramente la distinción del caso base y el inductivo.

Sin embargo ¿qué ocurre si n = 0? En ese contexto no se conseguiría llegar nunca al caso base y el método fallaría. Para corregir este problema, debemos comprobar que el número es mayor o igual a uno.

```
public static int s(int n) {
        int valor;
        if (n>0){
                if (n == 1){
                        valor = 1;
                }
                else {
                        valor = s(n-1) + n;
                }
        }
        else {
                System.out.println("El número no puede ser menor
que 1");
        }
}
```

Fig.15. Algoritmo 2. Suma recursiva con comprobación.

Mediante esta ampliación, si llamamos al método recursivo con un valor menor a 0, se consigue evitar el error y simplemente nos mostraría el mensaje de que no puede calcularlo.

9.1. Uso de drivers para el método recursivo

En el ejemplo previo, se comprueba c con el if(n > 0) si cumple la condición. Esta solución evita un error, pero añade complejidad al programa, ya que incluir esta comprobación en el método recursivo hace que se compruebe en todas las llamadas sucesivas al método recursivo, cuando es solo necesario comprobarlo la primera vez que se invoca.

Se trata de una situación normal en los métodos recursivos que se soluciona mediante el uso de un ***driver* encargado de comprobar la condición** para, una vez realizada, llamar al método recursivo.

El uso de un driver se puede implementar mediante un nuevo método o mediante un fragmento de código que se añade de manera previa a la llamada al método recursivo.

Si partimos del código mostrado en el algoritmo 2 y añadimos un nuevo driver, el resultado sería el mostrado a continuación:

A continuación podrás ver el recurso sobre el punto anterior:

```
class SumaRecursiva {

public static void main(String [] args) {
        int n;
        n=Integer.parseInt(args[0]);
        driverSuma(n);
} // Fin método main

// Método driver
public static void driverSuma(int n) {
        if (n>0) {
                System.out.print("Suma de los primeros "+ n+"
enteros:");
System.out.println(suma(n));
}
        else
                System.out.println("El número no puede ser <1");
} // Fin método suma

public static int suma(int n) {
        int valor;
        if (n == 1){
                valor = 1;
        }
        else {
                valor = suma(n-1) + n;
        }
}
```

Fig.16. Algoritmo 3. Driver.

/ 10. Resumen y resolución del caso práctico de la unidad

La aplicación de soluciones recursivas puede ser una de las mejores opciones cuando nos enfrentamos a resolver un determinado problema. Sin embargo, a pesar de ser una de las soluciones más elegantes que podemos implementar, hay que considerar, en todos los casos, el rendimiento que puede tener la solución recursiva. En ocasiones, podemos caer en una degradación del rendimiento de forma significativa.

A la hora de aplicar la recursividad, es importante determinar qué tipo vamos a escoger. Existen diferentes posibilidades a la hora de aplicar la recursividad, siendo posible que algunas de ellas generen situaciones en las que el programa aborte o se quede bloqueado por desbordamiento de pila.

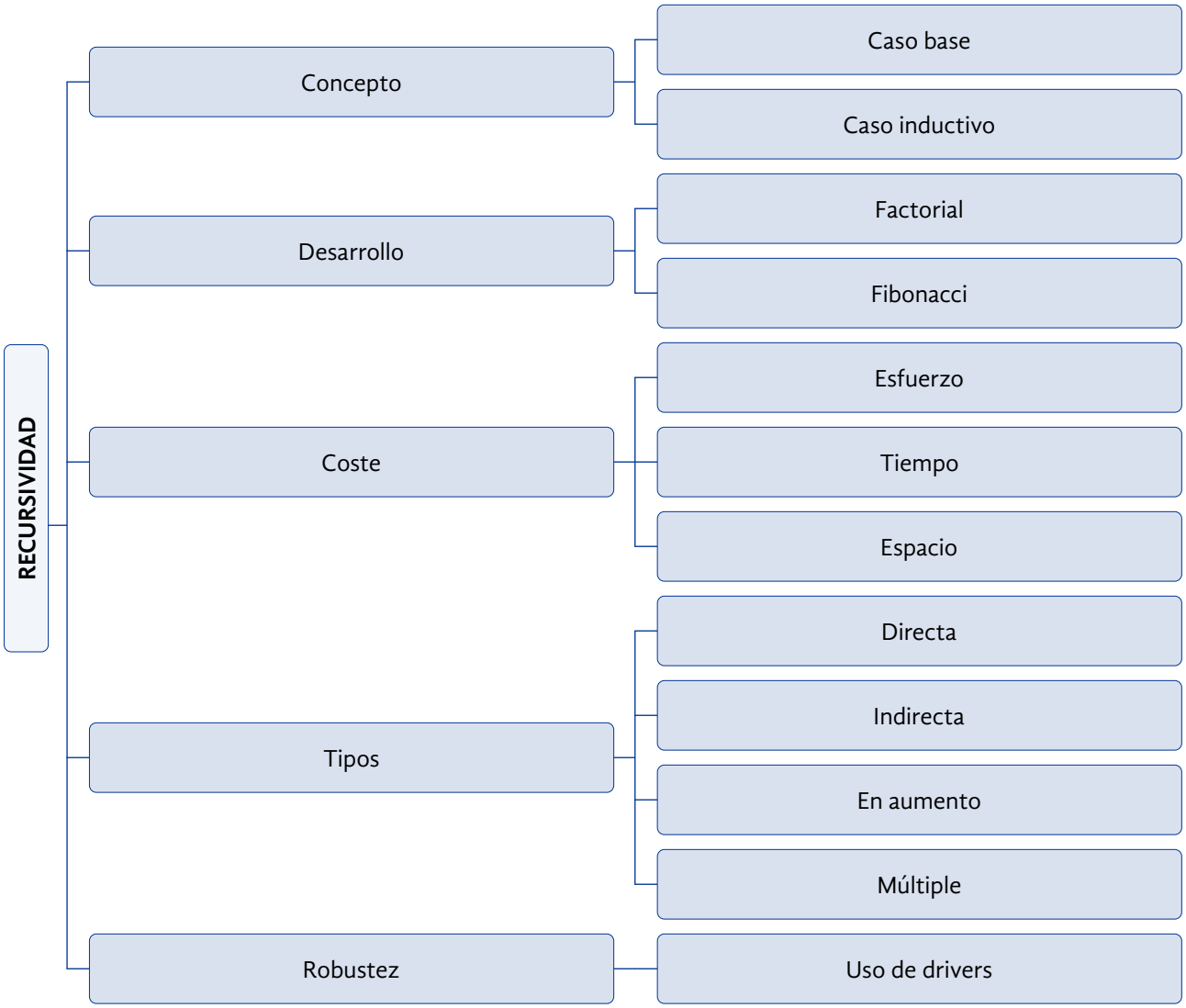

Fig.17. Esquema resumen del tema .

Resolución del caso práctico de la unidad

Cuando aplicamos un algoritmo recursivo, podemos elegir diferentes tipos de recursividad. La recursividad múltiple supone que se van a generar diferentes instancias del algoritmo N veces. En este caso, N determina el grado de la recursividad múltiple que estamos aplicando.

A medida que aumenta el valor de N, mayor número de recursos de la máquina vamos a necesitar para la ejecución del algoritmo. Si el valor de N es muy grande, se producirá un desbordamiento de pila, porque no quedarán recursos disponibles en la máquina para continuar la ejecución del algoritmo y resolver todas las llamadas recursivas pendientes. Aunque esto parezca una situación irreal, se suele producir mucho, en la realidad, cuando el algoritmo que estamos implementando aumenta el valor de entrada.

En este caso práctico, se está produciendo una situación similar a la que hemos descrito anteriormente. Para solucionar esto, es necesario analizar correctamente el grado de recursividad que estamos implementando y revisar el caso base.

/ 11. Bibliografía

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Haro, C. (2021). *Algoritmia: Razonar para crear*. Ediciones Eni.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Vicente, E. (2019). *Introducción a la programación informática: Una excelente guía visual que explica, paso a paso, desde el código binario hasta la creación de juegos*. Coordinación editorial de la edición española, Elsa Vicente. Dorling Kindersley.